EXPERIMENT 2

IMPLEMENTING THE PERCEPTRON ALGORITHM FOR FINDING THE WEIGHTS OF A LINEAR DISCRIMINANT FUNCTION.

Submission Date: 09 August, 2016

Student ID: 12.02.04.026
Section: A1
Year and Semester: 4.2
Ahsanullah University of Science and Technology
Department of Computer Science and Engineering

Contents

INTRODUCTION

One of the oldest algorithms used in machine learning is an online algorithm for learning weights of a linear discriminant function is called the Perceptron Algorithm.

The algorithm follows the following steps:

- Start with the all-zeroes weight vector w1 = 0, and initialize t to 1. Also let us automatically scale all examples x to have (Euclidean) length 1, since this does not affect which side of the plane they are on.
- Given example x, predict positive iff $wt \cdot x > 0$
- On a mistake, update as follows:

$$Mistake\ on\ positive: wt+1 \leftarrow wt+x.$$

$$Mistake\ on\ negative: wt+1 \leftarrow wt-x.$$

$t \leftarrow t+1$.

Thus if we make a mistake on a positive x we get $wt+1 \cdot x = (wt+x) \cdot x = wt \cdot x+1$, and similarly if we make a mistake on a negative x we have $wt+1 \cdot x = (wt-x) \cdot x = wt \cdot x-1$ So, in both cases we move closer to the value we want.

OBJECTIVE OF THE EXPERIMENT

The objective of the experiment is to Implement the Perceptron algorithm for finding the weights of a linear discriminant function. Given that the following sample points in a 2 class problem:

$\omega_1 = (1,1),(1,-1),(4,5)$
$\omega_2 = (2,2.5),(0,2),(2,3)$

1. Plot all sample points from both classes, but samples from the same class should have the same color and marker. Observer if these two classes can be separated with a linear boundary.

2. Consider the case of a second order polynomial discriminant function. Generate the high dimensional sample points y, as discussed in the class. We used the following formula:

$$y = [x_1^2 \quad x_2^2 \quad x_1 * x_1 \quad x_1 \quad x_2 \quad 1]$$

Also, normalize any one of the two classes.

3. Use Perceptron Algorithm (one at a time) for finding the weight-coefficients of the discriminant function (i.e., values of w) boundary for your linear classifier in question 2.
Here α is the learning rate and $0 < \alpha \leq 1$

$$W(i+1) = w(i) + \alpha y_m^(k)$$

If

$$w^T(i) * y_m^(k) \leq 0$$

i.e., if

$$y_m^(k)\ misclassified$$

$$w(i+1) = w(i)$$

if

$$w^T(i) * y_m^(k) > 0$$

4. Draw the decision boundary between the two classes using the values of w and the given MATLAB codes

PERFORMING THE EXPERIMENT

Plotting all sample points from the given classes

At first I plotted the given sample points of both classes ω_1 and ω_1 . The points of same class were plotted using the same color and marker. For ω_1 I used the color blue and the marker diamond whereas for ω_2 I used the color red and the marker star.

Matlab Code 1.1

```
class1 = [1 1; 1 -1; 4 5];
class2 = [2 2.5; 0 2 ; 2 3];
plot(class1(:,1), class1(:,2),'o','MarkerEdgeColor','b', 'MarkerFaceColor', 'b', 'MarkerSize',7);
grid on;
hold on;
plot(class2(:,1), class2(:,2),'s','MarkerEdgeColor','r', 'MarkerFaceColor', 'r', 'MarkerSize',5);
hold off;
```

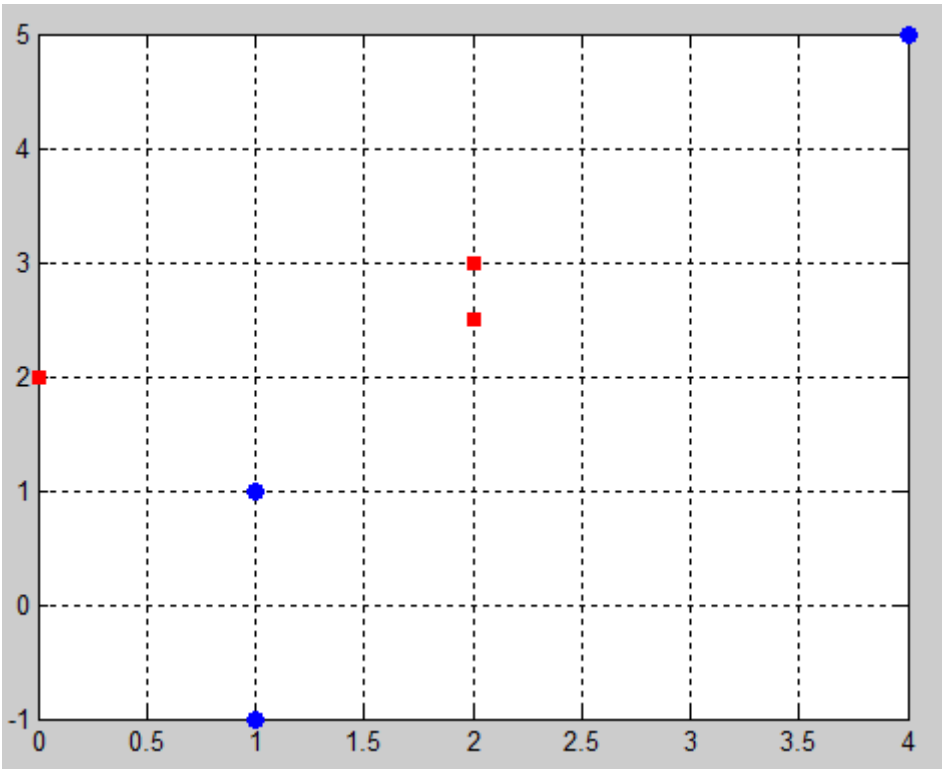

Figure 1: Output after running Matlab code 1.1

Generating y and normalizing any one of the two classes

The high dimensional sample points y are generated using the following formula: $y = [x_1^2 \quad x_2^2 \quad x_1 * x_1 \quad x_1 \quad x_2 \quad 1]$ (X_1, X_2, X_3, X_4) and then one of the two classes are normalized.

Matlab Code 1.2

```
y = zeros(6,6);
y = [
    class1(1,1)*class1(1,1)   class1(1,2)*class1(1,2)   class1(1,1)*class1(1,2)   class1(1,1)
class1(1,2) 1;
    class1(2,1)*class1(2,1)   class1(2,2)*class1(2,2)   class1(2,1)*class1(2,2)   class1(2,1)
class1(2,2) 1;
    class1(3,1)*class1(3,1)   class1(3,2)*class1(3,2)   class1(3,1)*class1(3,2)   class1(3,1)
class1(3,2) 1;

    -class2(1,1)*class2(1,1) -class2(1,2)*class2(1,2) -class2(1,1)*class2(1,2) -class2(1,1)
-class2(1,2) -1;
    -class2(2,1)*class2(2,1) -class2(2,2)*class2(2,2) -class2(2,1)*class2(2,2) -class2(2,1)
-class2(2,2) -1;
    -class2(3,1)*class2(3,1) -class2(3,2)*class2(3,2) -class2(3,1)*class2(3,2) -class2(3,1)
-class2(3,2) -1;
    ]
```

```
y =

    1.0000    1.0000    1.0000    1.0000    1.0000    1.0000
    1.0000    1.0000   -1.0000    1.0000   -1.0000    1.0000
   16.0000   25.0000   20.0000    4.0000    5.0000    1.0000
   -4.0000   -6.2500   -5.0000   -2.0000   -2.5000   -1.0000
         0   -4.0000         0         0   -2.0000   -1.0000
   -4.0000   -9.0000   -6.0000   -2.0000   -3.0000   -1.0000
```

Figure 2: Output after running Matlab code 1.2

Using Perceptron Algorithm (one at a time) for finding the weight-coefficients of the discriminant function

Matlab Code 1.3

```
w= ones(1,6);
alpha= 1/6;
flag = 0;
counter = 0;
while counter < 200
    counter = counter + 1;
    flag = 0;
    for i = 1:6
        g = y(i,:)*w' ;
        if g < 0
            w = w + alpha * y(i,:);
        else
            flag = flag + 1;
        end;
    end;
    if (flag == 6)
        break ;
    end;
end;
display(w);
display(counter);
```

```
w =

    9.0000   -5.8333    4.0000   -1.0000   -8.1667    2.8333

counter =

    97
```

Figure 3: Output after running Matlab code 1.3

Using Perceptron Algorithm (many at a time) for finding the weight-coefficients of the discriminant function

Matlab Code 1.4

```
w= ones(1,6);
alpha= 1/6;
flag = 0;
counter = 0;
temp = 0;
while counter < 200
    counter = counter + 1;
    flag = 0;
    temp = 0;
    for i = 1:6
        g = y(i,:)*w' ;
        if g < 0
            temp = temp + y(i, :);
        else
            flag = flag + 1;
        end;
    end;
    if (flag == 6)
        break ;
    end;
    w = w + alpha * temp;
end;
display(w);
display(counter);
```

```
w =

   10.8333   -7.3750    4.6667   -0.8333   -9.0833    2.0000

counter =

   102
```

Figure 4: Output after running Matlab code 1.4

Drawing the decision boundary

Matlab Code 1.5

```
1: syms x1 x2;
2: s=sym(w(1,1)*x1*x1+w(1,2)*x2*x2+w(1,3)*x1*x2+w(1,4)*x1+w(1,5)*x2+w(1,6));
3: s2=solve(s,x2);
4: xvals1=[-10:0.01:10];
5: xvals2(1,:)=subs(s2(2),x1,xvals1);
6: plot(xvals1,xvals2(1,:),'k');
7: grid, hold,
```

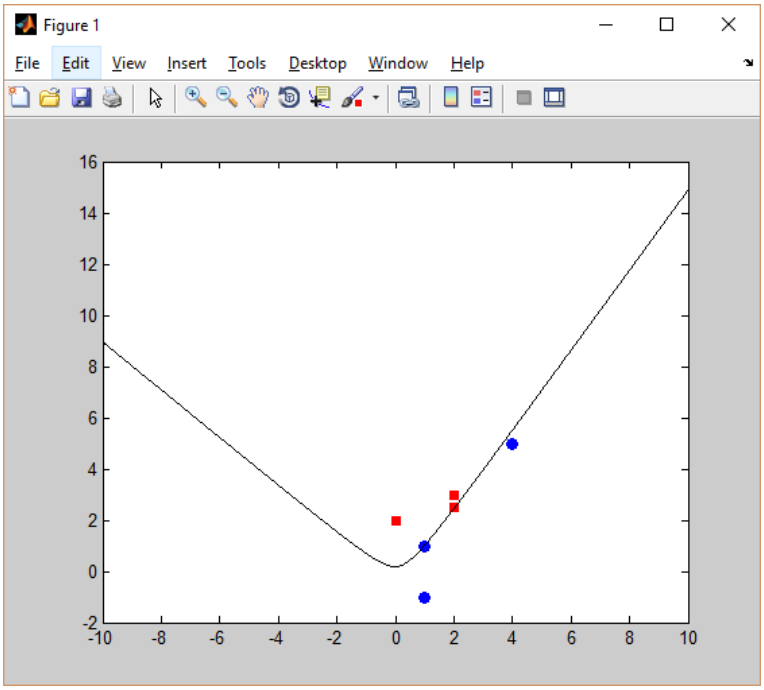

Figure 5: Output after running Matlab code 1.5

Result Analysis

In this experiment both one at a time and many at a time approach for Perceptron Algorithm was performed. We see here that the total number of loops used in each way is:

- One at a time: 97

- Many at a time: 102

As we can clearly see that many at a time takes much more time than one at a time. Here are only 6 datasets. It is possible that if there are much more dataset then one at a time would be much better than many at a time. The main reason of one at a time is better because it updates itself every time, where as many at a time do not.

CONCLUSION

We conclude from the experiment that one at a time approach of Perceptron algorithm is far better than the many at a time approach because in one at a time values are updated every time in contrast to many at a time.